

Python Time Module

```
# =====
# PYTHON DATE CLASS - COMPLETE EXPLANATION IN ONE SCRIPT
# =====

from datetime import date

# =====
# 1. TODAY'S DATE
# =====

today = date.today()

print("Today's date:", today)
print("Type:", type(today))

# Example output:
# Today's date: 2026-09-06
# Type: <class 'datetime.date'>

# =====
# 2. GET YEAR, MONTH, AND DAY
# =====

print("Year:", today.year)
print("Month:", today.month)
print("Day:", today.day)

# =====
# 3. CREATE A SPECIFIC DATE
# =====

birth_date = date(2001, 11, 4)

print("Birth date:", birth_date)
print("Year:", birth_date.year)
print("Month:", birth_date.month)
print("Day:", birth_date.day)

# =====
# 4. COMPARE DATES
# =====

date1 = date(2026, 9, 1)
date2 = date(2026, 9, 10)

print("date1:", date1)
print("date2:", date2)

print("date1 < date2:", date1 < date2)
print("date1 > date2:", date1 > date2)
print("date1 == date2:", date1 == date2)

# =====
# 5. CALCULATE DIFFERENCE BETWEEN DATES
# =====

start_date = date(2026, 9, 1)
end_date = date(2026, 9, 6)

difference = end_date - start_date
```

```

print("Difference:", difference)
print("Number of days:", difference.days)

# =====
# 6. ADD DAYS TO A DATE
# =====

from datetime import timedelta

today = date.today()

tomorrow = today + timedelta(days=1)
next_week = today + timedelta(days=7)
next_month_approx = today + timedelta(days=30)

print("Today:", today)
print("Tomorrow:", tomorrow)
print("Next week:", next_week)
print("After 30 days:", next_month_approx)

# =====
# 7. SUBTRACT DAYS FROM A DATE
# =====

yesterday = today - timedelta(days=1)
last_week = today - timedelta(days=7)

print("Yesterday:", yesterday)
print("Last week:", last_week)

# =====
# 8. FORMAT A DATE AS STRING
# =====

today = date.today()

formatted_date = today.strftime("%Y-%m-%d")

print("Formatted date:", formatted_date)

# Common formatting codes:
#
# %Y = Full year      → 2026
# %m = Month          → 09
# %d = Day            → 06
#
# %B = Full month     → September
# %b = Short month    → Sep
#
# %A = Full weekday   → Sunday
# %a = Short weekday  → Sun

print(today.strftime("%Y-%m-%d"))
print(today.strftime("%d/%m/%Y"))
print(today.strftime("%B %d, %Y"))
print(today.strftime("%A, %B %d, %Y"))

# =====
# 9. CONVERT STRING TO DATE
# =====

from datetime import datetime

```

```

date_string = "2026-09-06"

converted_date = datetime.strptime(
    date_string,
    "%Y-%m-%d"
).date()

print("String:", date_string)
print("Date:", converted_date)
print("Type:", type(converted_date))

# =====
# 10. GET WEEKDAY
# =====

today = date.today()

weekday_number = today.weekday()

print("Weekday number:", weekday_number)

# Monday    = 0
# Tuesday   = 1
# Wednesday = 2
# Thursday  = 3
# Friday    = 4
# Saturday  = 5
# Sunday    = 6

# =====
# 11. GET ISO WEEKDAY
# =====

print("ISO weekday:", today.isoweekday())

# Monday    = 1
# Tuesday   = 2
# Wednesday = 3
# Thursday  = 4
# Friday    = 5
# Saturday  = 6
# Sunday    = 7

# =====
# 12. CHECK IF A DATE IS BEFORE OR AFTER TODAY
# =====

deadline = date(2026, 9, 30)

if deadline > today:
    print("Deadline is in the future.")

elif deadline < today:
    print("Deadline has passed.")

else:
    print("Deadline is today.")

# =====
# 13. CALCULATE AGE
# =====

birth_date = date(2001, 11, 4)
today = date.today()

```

```

age = today.year - birth_date.year

# If birthday hasn't happened yet this year,
# subtract 1.
if (today.month, today.day) < (birth_date.month, birth_date.day):
    age -= 1

print("Age:", age)

```

```

# =====
# 14. DATA ENGINEERING EXAMPLE
# =====

```

Imagine an ETL pipeline receives an order date.

```

order = {
    "order_id": 1001,
    "customer": "Ahmed",
    "order_date": "2026-09-06"
}

order_date = datetime.strptime(
    order["order_date"],
    "%Y-%m-%d"
).date()

print("\nOrder date:", order_date)

```

Extract date parts for a data warehouse

```

order["year"] = order_date.year
order["month"] = order_date.month
order["day"] = order_date.day
order["weekday"] = order_date.strftime("%A")

print(order)

```

```

# =====
# 15. DATA WAREHOUSE DATE DIMENSION EXAMPLE
# =====

```

```

date_value = date(2026, 9, 6)

date_dimension_record = {
    "date": date_value,
    "year": date_value.year,
    "month": date_value.month,
    "day": date_value.day,
    "quarter": (date_value.month - 1) // 3 + 1,
    "day_name": date_value.strftime("%A"),
    "month_name": date_value.strftime("%B"),
    "is_weekend": date_value.weekday() >= 5
}

print("\nDate Dimension Record:")
print(date_dimension_record)

```

```

# =====
# 16. IMPORTANT DIFFERENCE
# =====

```

```

# date
# → stores only date
#
# datetime

```

```

# → stores date + time
#
# time
# → stores only time

only_date = date(2026, 9, 6)
date_and_time = datetime(2026, 9, 6, 14, 30, 0)

print("\nDate:", only_date)
print("Datetime:", date_and_time)

# =====
# SUMMARY
# =====

print("\n===== SUMMARY =====")

print("""
date.today()
    → Get today's date

date(YYYY, MM, DD)
    → Create a specific date

date.year
    → Get year

date.month
    → Get month

date.day
    → Get day

date1 - date2
    → Calculate date difference

date + timedelta(days=N)
    → Add days

date - timedelta(days=N)
    → Subtract days

date.strftime(...)
    → Date → String

datetime.strptime(...).date()
    → String → Date

date.weekday()
    → Monday=0 ... Sunday=6
""")

```